Spades Online: Product + Technical Spec

Owner: Demetrius Robinson (solo founder) **Audience:** Demetrius and any developer he hands this to **Version:** 0.1 draft, October 8, 2026 **Status:** Ready to start Phase 1. Open questions are listed in Section 15.

---

0. TL;DR

- **What:** A real-time multiplayer Spades game. It runs in the browser first, and native iOS/Android apps come later. It has a photoreal 3D table, friends, chat, leaderboards, and later live video.
- **Modes:** Joker Spades 2v2 (the default), Cutthroat (3 or 4 players, each for themselves), and Ace Down (52 cards, no jokers, aces high).
- **Budget:** $0 at launch. Everything runs on free tiers:
  - **Cloudflare Pages:** hosts the web app.
  - **Cloudflare Workers + Durable Objects:** run the game server, one Durable Object per table.
  - **Supabase:** accounts, friends, chat, stats, and leaderboards.
  - **WebRTC peer-to-peer:** video, with Cloudflare TURN as a relay fallback.
  - **Capacitor:** wraps the web app for the app stores later.
- **The server is the referee.** It shuffles, deals, checks every move, and never sends a player anyone else's cards.
- **Build order:** rules engine and bots first, then 2D online play, then social features, then the 3D look, then video, then native apps. The 3D look is deliberately *not* first, because a correct, fun, stable game matters more than graphics.

---

1. Goals and non-goals

Goals

1. People can play correct, fast Spades online with friends or strangers. The house rules are configurable, because every table plays a little differently.
2. It feels like sitting at a real table: a 3D felt table, real-looking cards, chat, and later faces on camera.
3. It costs nothing to run until it has real traction, and costs grow gradually and predictably after that.
4. One codebase covers web, iOS, and Android.

Non-goals (for now)

- Real-money play, wagering, or anything that could count as gambling. Coins or cosmetics with no cash value can be considered much later.
- Tournaments with prizes.
- Voice-only chat as a separate feature (video can carry audio; see Section 11).
- Spectator mode. It's a nice-to-have later and the protocol leaves room for it.

---

2. Users and core flows

Personas - *Regular group:* four friends who play Joker Spades every week. They need private tables, invite links, their own house rules, and chat or video. - *Solo player:* wants a quick game now. Needs quick match, bots to fill empty seats, and a rating that means something. - *Competitor:* wants to climb the leaderboards in their area and globally.

Core flows 1. **Sign up / sign in:** email magic link, Google, or (on iOS) Sign in with Apple. Pick a display name and avatar, confirm age (see Section 12), and optionally choose a region for local leaderboards. 2. **Play now (quick match):** pick a mode, get matched within N seconds, and bots fill any seats still empty after the wait. Bot-filled games are unrated by default. 3. **Private table:** create a table, choose a mode and house rules, share the invite link or code, friends join, and the host starts the game. 4. **In game:** bid, play tricks, see the score pad, chat, use emotes, and (later) turn on video. 5. **After the game:** final scores, rating change, stats update, a "Rematch" button, and "Add friend" for people you just played with. 6. **Social:** friends list with online/in-game presence, direct messages, join a friend's table, invite to a table. 7. **Leaderboards:** pick a mode and view global or local (by your chosen region) rankings, optionally per season.

---

3. Game rules

All rules live in one shared TypeScript package (`packages/rules`) that both the server and client use. **The server copy is the one that counts.** The client copy only highlights legal moves and animates instantly.

3.1 Rules common to all modes (unless a mode says otherwise)

Seats and dealing - Seats are numbered 0 to 3 clockwise. In partner modes, seats 0 and 2 are Team A and seats 1 and 3 are Team B. - The first dealer is random. The deal passes to the left (clockwise) each hand. - Cards are dealt one at a time clockwise, starting left of the dealer. Online, the server simply shuffles and splits the deck. - **Shuffle:** Fisher-Yates using `crypto.getRandomValues` with rejection sampling to avoid modulo bias. Optional "provably fair" mode: at the start of each hand the server publishes `SHA-256(deck order + secret salt)`, and reveals the salt and order after the hand so anyone can verify the deck wasn't changed mid-hand.

Trick play - The player to the dealer's left leads the first trick. House-rule option: the holder of the 2♣ leads it, which some tables use. - You must follow the suit that was led if you can. If you can't, you may play any card, including a trump. - Spades are always trump. In Joker modes, both jokers count as spades (see 3.2). - A trick goes to the highest trump played. If no trump was played, it goes to the highest card of the suit led. The trick winner leads the next trick. - **Breaking spades (on by default):** you can't lead a spade (or a joker) until a spade has been played on an earlier trick, unless your hand is only spades. - The server rejects illegal plays, so reneging can't happen and there are no renege penalties.

Bidding (after the deal, before the first lead) - Bidding starts left of the dealer and goes clockwise, with one bid each. - A bid is 0 to 13 tricks, or 0 to 17 in 3-player Cutthroat. A bid of 0 is **Nil**. - In partner modes, the two partners' bids add up to the team contract. A Nil bidder's partner makes their own bid alone. - **Nil:** the bidder promises to take zero tricks. The Nil is scored on its own, apart from the partner's contract. - **Blind Nil:** declared *before* seeing your cards. The UI holds your cards face down until you choose Blind Nil or "See cards." - House-rule settings: minimum team bid (default none; common values are 2, 4, or 5), whether Blind Nil is allowed (default: only when your team is behind by ≥100 points), the Blind Nil card exchange (default off; when on, partners swap 2 cards), and whether the partner of a Nil bidder may also bid Nil (default yes).

Scoring (defaults; all configurable)

Situation	Points
Contract made (tricks ≥ bid)	+10 × bid, plus +1 per extra trick ("bag")
Contract set (tricks < bid)	−10 × bid
Nil made	+100
Nil failed	−100
Blind Nil made	+200
Blind Nil failed	−200
Every 10 accumulated bags	−100, and the bag count drops by 10

- **Tricks taken by a failed Nil bidder:** by default they do *not* count toward the partner's contract, but they *do* count as bags for the team. This is a house rule: "count toward partner" or "ignore."
- **Game end:** first to reach the **target score (default 500; options 250 / 300 / 500 / custom)** at the end of a hand wins. If both sides pass the target in the same hand, the higher score wins. If they're exactly tied, play another hand.
- **Mercy / losing floor (default −200):** a side that falls to or below the floor loses immediately. Can be turned off.
- Optional house rules, off by default: **"10 for 200"** (bid 10 or more and make it for a double bonus), **bag penalty size** (−100 default, or −50), **bags per penalty** (10 default, or 5), **Moon / Boston** (take all 13 tricks for a bonus or instant win).

3.2 Mode 1: Joker Spades, 2v2 partners (default mode)

- **Players:** 4, in two partnerships sitting across from each other.
- **Deck:** 52 standard cards + Big Joker + Little Joker = 54, then **remove two low cards to get back to 52** (13 per player).
  - Default cards removed: **2♥ and 2♦**.
  - House-rule setting: `removedCards`, any 2 non-spade cards (common: 2♥+2♦, 2♣+2♦, 2♣+2♥).
- **Trump ranking, high to low (default): Big Joker > Little Joker > A♠ > K♠ > Q♠ > J♠ > 10♠ … 3♠ > 2♠**
- **House-rule setting `deuceOfSpadesHigh` (default off):** when on, the 2♠ ranks just under the jokers: Big Joker > Little Joker > 2♠ > A♠ > K♠ …
- **Advanced setting `extraHighTrumps` (default empty):** some tables also promote the 2♦ (for example Big Joker > Little Joker > 2♦ > 2♠ > A♠…). The engine models the trump order as an ordered list, so any local variant is a config change, not a code change. The 2♦ then counts as a spade for following suit, and it can't also be one of the removed cards.
- **Jokers count as spades** for following suit, for trumping, and for the "spades broken" rule. Leading a joker counts as leading a spade.
- **Non-trump suits** rank A high down to 2, minus any removed cards.
- Bidding and scoring follow 3.1.
- **Display:** a "Big" and "Little" joker that are clearly different (color vs. black-and-white, as on real decks), plus a little legend at the table showing that table's trump order.

3.3 Mode 2: Cutthroat (individual, 3 or 4 players)

- **No partners.** Everyone bids and scores for themselves, and bags are tracked per player.
- **4 players:** 13 cards each. The deck follows the table's deck setting: either the Joker deck from 3.2 (54 − 2 = 52) or a standard 52 with no jokers. **Default: Joker deck**, since Joker is the group's usual style. (Open

question 15.3.)

- **3 players:** 17 cards each, so the deck needs 51 cards.
  - Standard deck: remove **2♣** (52 − 1 = 51).
  - Joker deck: remove **three** low non-spade cards (54 − 3 = 51). Default: 2♣, 2♥, 2♦.
  - Both are configurable via `removedCards`.
- Bids run 0 to 13 (4p) or 0 to 17 (3p). Nil and Blind Nil are allowed by default. With no partner, the Blind Nil exchange doesn't apply, and the default "behind by ≥100" condition compares you with the leader.
- Scoring follows 3.1, applied per player. **Default target 300** for Cutthroat, since games run long with individual scores; configurable.
- **Placement:** when the game ends, players are ranked by final score, with ties sharing a place. This placement feeds the ratings (Section 9).
- **Seat-filling:** a 3-player table can't take a 4th player mid-game. In quick match, bots fill only to the table's chosen size.

3.4 Mode 3: Ace Down

Confirmed by Demetrius: standard 52-card deck, **no jokers**, and **aces are the highest cards** (A♠ is the top trump; A > K > Q > … > 2 in every suit).

Nothing else was specified, so **until confirmed, Ace Down uses the standard rules in 3.1:** - 2v2 partners by default. A host could also run it as Cutthroat, since the deck and ranking settings are independent of the partnership setting. - Standard bidding with Nil and Blind Nil, standard scoring, bags at 10 = −100, target 500, floor −200. - `deuceOfSpadesHigh` is forced off, and there's no card removal.

> ⚠️ Bidding and scoring for Ace Down need confirming with Demetrius. The name suggests there may be a regional twist (for example a special rule about aces, a required minimum bid, or a different way of handling Nil). See Open question 15.1. The engine treats Ace Down as a **preset** of settings, so any confirmed differences become config changes or one small rule hook.

3.5 House-rule settings (one config object)

```
type TableRules = {
  mode: 'joker' | 'cutthroat' | 'aceDown';
  partnerships: boolean;              // joker/aceDown: true; cutthroat: false
  playerCount: 3 | 4;                 // 3 only when partnerships=false
  deck: 'joker54' | 'standard52';
  removedCards: Card[];               // e.g. ['2H','2D']; length is checked against playerCount
  trumpOrderTop: Card[];              // e.g. ['BJ','LJ'] or ['BJ','LJ','2S']; above A♠
  deuceOfSpadesHigh: boolean;         // shortcut that inserts '2S' after the jokers
  spadesMustBeBroken: boolean;        // default true
  firstLead: 'leftOfDealer' | 'twoOfClubs';
  minTeamBid: number;                 // default 0 (no minimum)
  nil: { allowed: boolean; value: number };            // default true, 100
  blindNil: { allowed: boolean; value: number; onlyWhenBehindBy: number | null; exchangeCards: 0 | 1 |
2 };
  nilTricksCountForPartner: boolean;  // default false (count as bags)
  bagsPerPenalty: number;             // default 10
  bagPenalty: number;                 // default 100
  tenForTwoHundred: boolean;          // default false
  targetScore: number;                // default 500 (cutthroat 300)
```

```
  losingFloor: number | null;         // default -200
  turnTimerSec: { bid: number; play: number };  // default 30 / 20
};
```

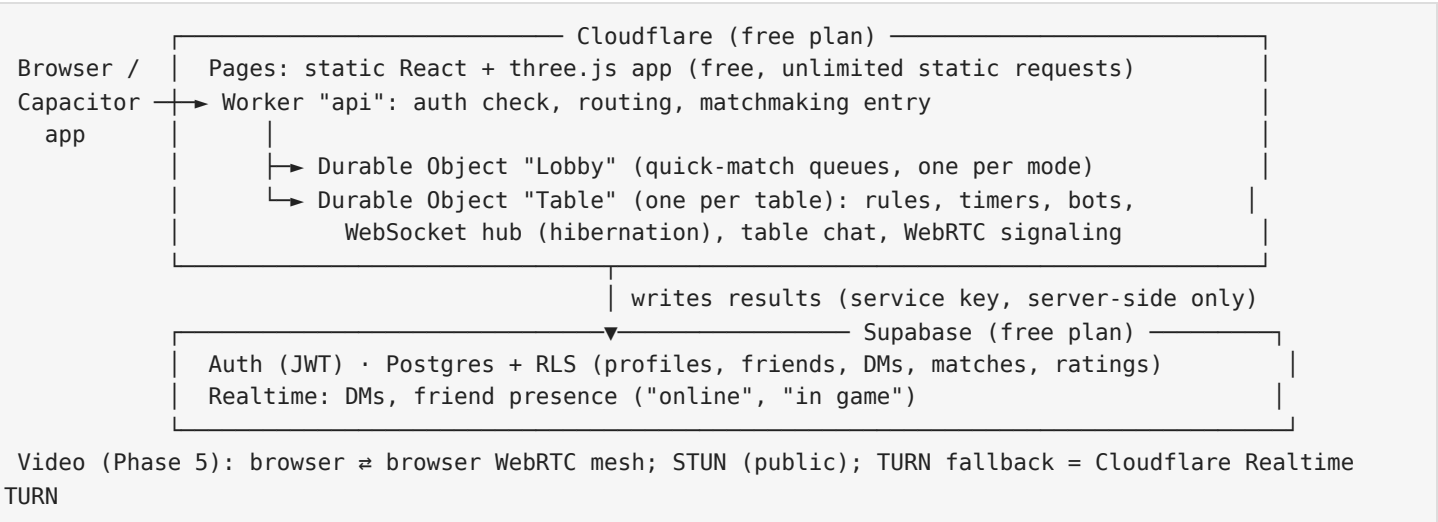
- **Ranked games use a fixed preset per mode**, so ratings compare like with like. Private tables can use any settings, and those games are unrated unless the settings match the ranked preset.
- The server checks every config. For example, with 4 players `removedCards` has to leave exactly 52 cards, and with 3 players exactly 51.

3.6 Bots

- Bots are needed in Phase 1, both for testing and for filling seats.
- **v1, heuristic:**
  - *Bidding:* count sure tricks (top trumps, guarded aces and kings, voids/singletons with trump to cut). Bid Nil when the hand is very low with no high spades.
  - *Play:* follow standard play conventions. Cover your partner's Nil. Duck to avoid bags when the contract is already made. Set the opponents when you can.
- **v2, later:** Monte Carlo "determinization." The bot samples possible hidden hands that fit what it has seen and picks the best average move. Keep compute small on the Workers Free plan; check current CPU limits (Section 13).
- **Bots only see what a human in that seat would see.** They run inside the table's Durable Object and get exactly that seat's view. No peeking.
- Bot difficulty levels: Easy, Normal, Hard.

---

4. Architecture



4.1 Frontend

- **Stack:** TypeScript, React, Vite, three.js through **react-three-fiber** and **drei**, Zustand for state, and Zod (v4.5+; Cloudflare notes that older versions use much more memory) for message schemas shared with the server.
- **Hosting:** Cloudflare Pages, deployed from GitHub.
  - Limits: 500 builds/month, 20,000 files, 25 MiB per file. Keep each 3D asset under 25 MiB, which is easy with compression.

- **2D fallback renderer:** a simple DOM/SVG table used in Phase 1, kept forever as the "Low / Classic" mode and for accessibility.

4.2 Game server: Cloudflare Workers + Durable Objects

- **Worker `api`:**
  - Checks the Supabase JWT, using the project's JWKS / signing keys (or the legacy JWT secret, depending on project settings).
  - Creates tables and routes `wss://…/table/:id` to that table's Durable Object.
- **Durable Object `Table` (one per table):** the single source of truth for that game.
  - Holds the game state and runs the rules engine on every action.
  - Uses the **WebSocket Hibernation API** (`ctx.acceptWebSocket`), so an idle table is *not* billed for wall-clock time while players think. Uses `setWebSocketAutoResponse` for ping/pong, which Cloudflare says costs no extra duration.
  - Saves state to its **SQLite storage** (the only Durable Object storage the free plan allows), so a hibernated or restarted object picks up exactly where it was.
  - Runs **turn timers with a Durable Object alarm**. There's one alarm per object, set to the earliest deadline. When it fires, a bot makes the move for the absent player.
  - Runs bots in-process.
  - Relays table chat and WebRTC signaling.
  - When the game ends, it writes the match result to Supabase and computes the rating update (Section 9).
- **Durable Object `Lobby` (one per mode):** quick-match queue.
  - Groups players by rating band and widens the band over time.
  - After a timeout (default 20 s), creates a table and fills the rest with bots, marking the game unrated if bots are seated.
- **Choosing location:** create the Table object with a `locationHint` near the host or the first player in the queue.
- **Saving state economically:** on the free plan, *rows written* (100k/day) is probably the first limit you'll hit (see Section 13).
  - Store the state as **one row that gets overwritten** (1 row write per action), not one row per event.
  - Keep the event log for the current hand in memory, plus a compact snapshot. Don't keep a growing table.

4.3 Accounts, social, and stats: Supabase

- **Auth:** email magic link and OAuth (Google; Apple, which iOS will require when other social logins are offered).
- **Postgres** with **row-level security on every table** (Section 6).
- **Realtime:**
  - DMs: Postgres Changes on `messages`, or Broadcast with the message also saved to the table.
  - Friend presence: one presence channel per user. Friends subscribe to it.
  - Keep **one Realtime connection per client** and multiplex channels over it. The free plan allows 200 concurrent connections (Section 13).
- **Table chat goes through the Table Durable Object, not Supabase,** to save Supabase quota. The object keeps the last ~100 lines in memory so they can be attached to a report.
- **Free project pausing:** Supabase's pricing page says free projects **pause after 1 week of inactivity**. Real players keep it active, but watch out for this during quiet development weeks. Unpausing is manual from the dashboard.

4.4 Video: WebRTC mesh

- Covered in Section 11.

4.5 Native apps: Capacitor

- Wrap the same web build. Add native plugins as needed: push notifications, haptics, camera permissions, in-app review.
- Check that WebGL2 performance is acceptable in iOS WKWebView and Android WebView, especially with the Low/Medium quality tiers.

4.6 Repo layout (monorepo, pnpm workspaces)

```
packages/rules      # pure TS rules engine, seeded RNG, no I/O; 100% unit tested
packages/bots       # bot strategies (use rules)
packages/protocol   # Zod schemas + TS types for all WebSocket messages
packages/rating     # Glicko-2 implementation + tests
apps/web            # React app (2D + 3D renderers)
apps/server         # Cloudflare Worker + Durable Objects (wrangler)
supabase/           # SQL migrations, RLS policies, seed data
assets-src/         # Blender files, raw textures (not deployed)
```

---

5. Photoreal 3D table and cards (Phase 4)

Look target: a warm room, a wood-rimmed felt table under soft overhead light, and cards with slight thickness, a satin sheen, and subtle wear.

Assets - **Table:** modeled in Blender with a wood rail and felt surface. Use **CC0 PBR textures from Poly Haven** (wood, felt/fabric; license: CC0). Bake ambient occlusion (and optionally a lightmap) for the table in Blender, so it looks rich without real-time cost. - **Cards:** a single shared mesh, a thin rounded rectangle with a tiny bevel. - Faces come from **one texture atlas** (52 faces + 2 jokers + back), plus a roughness map for a satin finish and slight edge wear. - Card art has to be original or properly licensed (Open question 15.6). A four-color deck option is good for accessibility. - **Environment:** an **HDRI from Poly Haven** (CC0) for image-based lighting and reflections, plus one shadow-casting light. Use a 1k HDRI on low quality and 2k on high. - **Rendering:** PBR materials (`MeshStandardMaterial` / `MeshPhysicalMaterial` for the card sheen), ACES or AgX tone mapping, sRGB output, and drei `ContactShadows` or soft shadow maps. Optional post-processing on High (SSAO, subtle depth of field on the far side of the table).

Compression / delivery - Models: glTF/GLB, compressed with **Draco** (or meshopt) via `gltf-transform`. - Textures: **KTX2 / Basis Universal**. Use UASTC for card faces so the text stays sharp, and ETC1S for wood and felt. Load them with three.js `KTX2Loader`. - Budget: aim for **≤ ~8–15 MB on first load at Low/Medium**, and lazy-load High textures. Static files on Cloudflare Pages are free to serve, but every file must be ≤ 25 MiB.

Quality settings (chosen automatically, with a manual override)

Setting	Low (old phones)	Medium	High
Device pixel ratio cap	1.0	1.5	2.0
Texture size	512–1k	1k–2k	2k (4k felt optional)

Setting	Low (old phones)	Medium	High
Shadows	baked only	contact shadows	soft shadow map
Post-processing	none	none	SSAO, light DoF
HDRI	1k, no background	1k	2k
Antialiasing	none / FXAA	MSAA ×2	MSAA ×4

- Auto-detection uses drei `PerformanceMonitor`. If the frame rate stays under ~30 fps, it drops a tier. If it's still slow at Low, it offers the 2D Classic table.
- **Interaction:** your hand is fanned at the bottom of the screen in screen space. Tap a card to raise it and tap again to play (or drag it to the center). Illegal cards are dimmed. Played cards animate to the center, and won tricks slide to the winner's pile. Pause rendering when nothing is moving (`frameloop="demand"`) to save battery.

---

6. Data model (Supabase Postgres)

RLS is **on for every table**. The Table Durable Object writes match results with the **service role key**, which is stored only as a Worker secret and never shipped to clients.

```
-- Users -----------------------------------------------------------------
profiles (
  id uuid pk references auth.users,        -- 1:1 with auth user
  username citext unique not null,          -- display handle
  display_name text, avatar_url text,
  birth_year int,                           -- age gate (or store a flag, not a full DOB)
  is_minor boolean not null default false,  -- 13–17
  region_code text references regions,      -- user-chosen, for local boards (no GPS)
  video_enabled boolean default false,      -- adult + opted in
  status text default 'active',             -- active | muted | suspended | banned
  created_at timestamptz default now()
)
regions (code text pk, name text, country_code text, parent_code text)  -- e.g. 'US-GA', 'US-GA-ATL'

-- Social ----------------------------------------------------------------
friendships (
  user_low uuid, user_high uuid,            -- ordered pair: one row per pair
  requested_by uuid, status text,           -- pending | accepted
  created_at timestamptz, accepted_at timestamptz,
  primary key (user_low, user_high)
)
blocks (blocker uuid, blocked uuid, created_at timestamptz, primary key (blocker, blocked))
conversations (id uuid pk, kind text /* dm */, created_at timestamptz)
conversation_members (conversation_id uuid, user_id uuid, last_read_at timestamptz,
pk(conversation_id,user_id))
messages (
  id bigint identity pk, conversation_id uuid, sender_id uuid,
  body text check (char_length(body) <= 500), created_at timestamptz,
  deleted_at timestamptz                    -- soft delete for moderation
)

-- Games -----------------------------------------------------------------
matches (
  id uuid pk, mode text, rules jsonb, rated boolean,
  started_at timestamptz, ended_at timestamptz,
```

```
  end_reason text,                          -- target | floor | abandoned
  hands_played int, season_id int
)
match_players (
  match_id uuid, seat smallint, user_id uuid null,   -- null = bot
  bot_level text null, team smallint null,
  final_score int, placement smallint, bags int,
  bids_made int, bids_set int, nils_made int, nils_failed int,
  left_early boolean default false,
  primary key (match_id, seat)
)
match_hands (match_id uuid, hand_no int, summary jsonb, primary key(match_id, hand_no)) -- optional,
for replays; watch the 500 MB DB cap

-- Ratings and stats ---------------------------------------------------
seasons (id serial pk, name text, starts_at timestamptz, ends_at timestamptz)
ratings (
  user_id uuid, mode text, season_id int,   -- season_id 0 = all-time
  rating float8 default 1500, rd float8 default 350, volatility float8 default 0.06,
  games int default 0, wins int default 0, last_played_at timestamptz,
  primary key (user_id, mode, season_id)
)
rating_history (user_id uuid, mode text, match_id uuid, before float8, after float8, rd_after float8,
created_at timestamptz)
user_stats (user_id uuid, mode text, games int, wins int, avg_bid numeric, bid_accuracy numeric,
            nil_attempts int, nil_success int, total_bags int, primary key(user_id, mode))

-- Safety -------------------------------------------------------------
reports (
  id bigint pk, reporter uuid, reported uuid, context text,  -- dm | table_chat | video | name
  match_id uuid null, evidence jsonb,       -- last N chat lines, captured server-side
  reason text, status text default 'open', created_at timestamptz, resolved_by uuid, notes text
)
```

RLS sketch - `profiles`: anyone signed in can read public columns, through a `public_profiles` view that hides `birth_year`. Users can update only their own row, and can't change `status`. - `friendships`: you can see and modify only rows you're part of. Inserting is allowed only if neither side has blocked the other. - `messages`: read and insert only when you're a member of that conversation and not blocked. Enforce this with a `security definer` function. - `matches`, `match_players`, `ratings`: readable by all signed-in users. **Only the service role can write them.** - `reports`: insert your own. Only admins can read.

Leaderboard queries - Global: `ratings` filtered by mode and season, `games >= 10`, sorted by **conservative rating = rating − 2·RD**. - Local: the same query, joined to `profiles.region_code`. - To stay fast and cheap, a cron job refreshes a materialized view every few minutes. A Cloudflare Cron Trigger or Supabase `pg_cron` would work; check that `pg_cron` is available on the free plan.

---

7. Real-time protocol (client ⇄ Table Durable Object)

- **Transport:** one WebSocket per player per table: `wss://api.<domain>/table/<tableId>?token=<supabase JWT>`. The token can go in the first message instead of the URL so it doesn't show up in logs.
- **Format:** JSON validated with Zod. Every server message has a monotonically increasing `seq`, so clients can spot gaps and resync.

Client → server

type	payload	notes
`hello`	`{ token, lastSeq? }`	Authenticate. Reconnects send `lastSeq`.
`sit` / `stand`	`{ seat }`	Lobby phase only.
`setRules`	`{ rules }`	Host only, before start.
`start`	`{}`	Host; empty seats get bots.
`bid`	`{ handNo, bid: number | 'nil' | 'blindNil' }`	Validated for turn, range, and house rules.
`revealCards`	`{ handNo }`	Opts out of the Blind Nil option.
`exchange`	`{ handNo, cards }`	Blind Nil exchange, if enabled.
`play`	`{ handNo, trickNo, card }`	Rejected if not your turn or not legal.
`chat`	`{ text }`	≤ 200 characters, rate-limited.
`emote`	`{ id }`	Pre-set reactions.
`rtc`	`{ to: seat, kind: 'offer'|'answer'|'ice', data }`	Video signaling relay (Phase 5).
`rematch` / `leave`	`{}`	

Server → client

type	payload	notes
`welcome`	`{ you: {seat,userId}, table: PublicState, hand?: Card[], seq }`	Full snapshot: on join, on reconnect, or when a gap is detected.
`seated` / `playerStatus`	`{ seat, user?, bot?, connected }`	
`handStart`	`{ handNo, dealer, deckCommit? }`	Sent to all players.
`yourHand`	`{ handNo, cards }`	**Sent only to that player.**
`turn`	`{ seat, phase: 'bid'|'play', deadline }`	`deadline` is a server timestamp.
`bidMade`	`{ seat, bid }`	
`cardPlayed`	`{ seat, card, trickNo }`	A card is revealed only when it's played.
`trickWon`	`{ trickNo, winner }`	
`handScored`	`{ handNo, perSide: {bid, tricks, points, bags, nil?}[], totals }`	
`gameOver`	`{ winner/placements, finalScores, ratingDeltas?, matchId }`	
`chat` / `emote`	`{ seat, text/id, ts }`	
`rtc`	`{ from: seat, kind, data }` / `iceServers`	Short-lived TURN credentials.
`error`	`{ code, message, ref? }`	Examples: `NOT_YOUR_TURN`, `ILLEGAL_CARD`, `MUST_FOLLOW_SUIT`, `SPADES_NOT_BROKEN`.

Connection rules - **Heartbeats:** the client pings every ~20–30 s and the server auto-responds, which keeps hibernation working. If a socket closes, the player is marked `connected:false` and the others see a "reconnecting…" badge. - **Reconnection:** - The client retries with exponential backoff (0.5 s → 8 s) and sends `hello{lastSeq}`. - The server replies with a fresh `welcome` snapshot, including your hand. Snapshots are small, so don't bother replaying

events. - Seats are held for the whole game. If you're disconnected **on your turn**, the timer keeps running and a bot plays for you when it expires. - After **3 consecutive auto-moves, or 3 minutes away**, a bot takes the seat until you return. - In rated games, leaving for good counts as a loss for rating purposes and sets `left_early`. - **Turn timers:** default 30 s to bid and 20 s to play. Optionally add a 60 s "time bank" per game. The UI shows a countdown ring using the server's `deadline`, adjusted for clock offset measured at `hello`. - **Idempotency:** `play` and `bid` include `handNo`/`trickNo`, so a duplicate resend after reconnecting is safely ignored.

---

8. Anti-cheat basics

1. **Server-authoritative.** The client only sends *intentions* like "play Q♥." The server decides whether that's legal and what happens next.
2. **The server never sends other players' hands.** Each player gets only `yourHand`. Other cards are revealed one at a time through `cardPlayed`. No hidden data reaches the client, so there's nothing to dig out of memory or DevTools. Bots get the same restricted view.
3. **Secure shuffle** (crypto RNG). Optional deck commitment (hash before, reveal after) so fairness can be checked.
4. **Validate everything:** check every message against its Zod schema, check turn order and legality, and rate-limit each socket (for example ≤ 10 msgs/s, and chat ≤ 1 msg/s with bursts of 3).
5. **Collusion:** this is the real risk in Cutthroat and in rated games, for example friends on a call sharing their hands.
    - Rated quick match pairs strangers, and friends can't queue together into the same rated Cutthroat table.
    - Games with video or private tables are unrated by default.
    - Flag accounts that keep ending up at the same tables, or that play suspiciously (for example deliberately feeding tricks).
6. **Rating abuse:** minimum games before appearing on leaderboards, unrated bot games, and abandonment counted as a loss. Watch for many new accounts from the same IP all playing each other; IP is a weak signal, so use it only as a flag.
7. **Secrets:** the Supabase service key and TURN API token live only in Worker secrets. Clients get only the Supabase anon key, which RLS protects, and short-lived TURN credentials.

---

9. Scoring, ratings, and leaderboards

- **In-game scoring:** Section 3.
- **Rating system: Glicko-2** (Glickman's paper). Each player has a separate rating per mode: `joker_2v2`, `cutthroat_3`, `cutthroat_4`, `ace_down`.
    - Start values: rating 1500, RD 350, volatility 0.06, system constant $\tau \approx 0.5$.
    - Update after every rated game. Glicko-2 is designed for rating periods, so treating each game as its own period is a simple, common approximation.
    - **2v2:** each player's opponent is a composite built from the opposing team's average rating, with RD combined as `sqrt((RD1²+RD2²)/2)`. Result is 1 or 0. Both partners get an update, so a strong player with a weak partner gains more for a win.
    - **Cutthroat:** split the final placement into pairwise results (each pair of players counts as one "game": win, loss, or 0.5 for a tie) and update each player against all the others in one period.
    - RD grows with inactivity (the standard Glicko-2 step), so returning players move faster.
- **Leaderboards**

    - **Global** per mode, plus **Local** per user-chosen region (country → state/province → optional metro, picked from a list; no GPS or IP lookup needed).
    - Users can change region at most once every 30 days, to stop people hopping regions.
    - Sorted by **conservative rating (rating − 2·RD)** with a minimum of 10 rated games. Also show win %, bid accuracy, and Nil success rate.
    - Friends leaderboard: an easy add, filtering the same query by friend IDs.
- **Seasons (optional, Phase 3+):** about 3 months each. At season start, keep the all-time rating and start a season row with rating pulled toward 1500 and RD reset higher (around 200). Give end-of-season badges.
- **Stats shown on profile:** games, wins, win %, average bid, contracts made %, Nils tried/made, bags per game, current/best rating per mode, recent matches.

---

10. Friends and chat

- **Friend requests:** by username, by invite link, or from the post-game screen. Accept or decline. Removing a friend is silent.
- **Presence:** offline, online, or in game (mode, plus a "join" button if the table has an open seat and the friend allows it).
    - Presence is published on a Supabase Realtime presence channel per user.
    - The Supabase free plan limits presence to 20 messages/sec project-wide, so throttle presence updates; Section 13 links the exact limits. This is a reason to move presence to a Durable Object later if it becomes a bottleneck.
- **DMs:** one-to-one, text only at first, max 500 characters, kept for 90 days. Read receipts are optional.
- **Table chat:** goes through the Table Durable Object, isn't stored after the game except as report evidence, and has quick emotes. A "mute player" control is on every seat.
- **Group chats:** later. The schema already supports them through `conversations`.

---

11. Live video (stretch goal, Phase 5)

- **Topology:** full mesh with up to 4 players. Each browser sends its camera to the other 3 and receives 3.
    - Keep video small and light, for example 320×240 or 480×360 at 15–24 fps. Lower the resolution and bitrate automatically when more people are on camera, and offer "audio only" and "camera off."
    - Mesh is fine at 4 people. Bigger rooms would need an SFU, which costs money.
- **Signaling:** goes through the **Table Durable Object** using the `rtc` messages. Signaling is a handful of messages per peer pair, so it barely touches quota.
- **NAT traversal:**
    - **STUN:** a public STUN server such as `stun.cloudflare.com:3478`.
    - **TURN fallback: Cloudflare Realtime TURN**. The Worker mints **short-lived TURN credentials** through Cloudflare's API and sends them in `iceServers`, so the API token never reaches clients.
    - Pricing: Cloudflare charges **$0.05 per GB** of egress for SFU and TURN, with **the first 1,000 GB each month free** (shared between SFU and TURN). TURN is free when used with Cloudflare's SFU. Only connections that can't go direct use TURN.
    - Per-user limits are listed in the TURN docs.
- **Defaults and safety:**
    - Video is **off by default** and opt-in per table.
    - **Adults only (18+)**, and only at private tables or tables where everyone has video turned on.
    - Anyone can hide anyone else's video on their own screen.

    - Video is peer-to-peer, so **nothing is recorded or stored on our servers**. The report flow records who, when, and which table, plus the reporter's description.
    - Video tables are unrated by default (collusion risk).
- **Mobile:** camera and mic permissions in Capacitor. Pause video when the app goes to the background.

---

12. Moderation, safety, and privacy

- **Minimum age:**
    - **13+ to make an account.** Ask for age at signup and block users under 13. Collecting data from under-13s brings in COPPA parental-consent requirements, which are out of scope.
    - **13–17:** chat only with accepted friends, no DMs from strangers, no video, and profiles hidden from search.
    - **18+:** everything.
    - Have a lawyer review the age policy and Terms before a public launch (Open question 15.8).
- **Report and block** are available everywhere a user can see another user: profile, table seat, DM, chat line, and video tile.
    - **Block:** they can't DM you, friend you, or see your presence. Quick match tries not to seat you together, and you never share a private table.
    - **Report:** pick a reason (harassment, hate, sexual content, cheating, underage, other). The server attaches the last ~50 table-chat lines or DM messages as evidence.
- **Filters:**
    - A profanity/slur filter on usernames, chat, and DMs (a word list on the server, with "mask" or "block" behavior). Users can choose a "family filter" on or off; for minors it's always on.
    - Links are blocked in chat.
    - Rate limits to stop spam.
- **Enforcement:** profile `status` can be `muted` (can't chat), `suspended` (until a date), or `banned`. To start, Demetrius reviews reports in the Supabase dashboard or a tiny admin page. Repeat offenders get video and chat removed first.
- **App store requirements (Phase 6):**
    - Apps with user-generated content must offer filtering, reporting, blocking, and published contact info. See Apple App Review Guidelines 1.2.
    - Apps that let users create accounts must also let them delete their account in-app.
    - Google Play has similar rules.
- **Privacy:** collect as little as possible. Region is user-chosen, there's no GPS, and storing birth year or an age band beats storing a full date of birth. Publish a privacy policy and terms, and offer data export and deletion.

---

13. Free-tier limits (verified October 8, 2026) and where costs start

Limits change. **Re-check these pages before launch.** All figures below come from the official pages linked.

13.1 Verified free-tier limits

Service	Free tier (as published)	What happens at the limit	Source
Cloudflare Pages (static hosting)	Static asset requests **free and unlimited**. 500 builds/month, 1 concurrent build, 20,000 files/site, 25 MiB max file.	Builds queue or fail at the build limit.	Pages limits, Pages Functions pricing
Cloudflare Workers	**100,000 requests/day** (all Workers + Pages Functions combined, reset 00:00 UTC), **10 ms CPU per invocation**, 128 MB memory. A WebSocket counts as one request (the upgrade). Messages routed through a Worker don't count.	Error 1027 (or "fail open") until the reset.	Workers pricing, Workers limits
Durable Objects (free plan, SQLite only)	**100,000 requests/day**; incoming WebSocket messages count **20:1** (20 messages = 1 request), outgoing messages are free. **13,000 GB-s/day** duration. SQLite: **5M rows read/day, 100,000 rows written/day, 5 GB total storage**. Hibernating idle objects aren't billed for duration.	"Further operations of that type will fail with an error" until 00:00 UTC.	Durable Objects pricing, Durable Objects limits
Supabase Free	**50,000 MAU**, **500 MB database**, **5 GB egress** (+5 GB cached egress), 1 GB file storage, 500,000 Edge Function invocations, **Realtime: 200 concurrent connections, 2M messages/month, 100 msgs/sec, 20 presence msgs/sec**, 256 KB broadcast payload. **2 active free projects. Paused after 1 week of inactivity.** No automatic backups. 1-day log retention.	Realtime refuses new connections over 200 (`too_many_connections`). Over-quota usage may get the project restricted.	Supabase pricing, Realtime limits
Cloudflare Realtime TURN	**First 1,000 GB/month of egress free** (shared with the SFU), then **$0.05/GB**. TURN is free when used with Cloudflare's SFU. STUN at `stun.cloudflare.com`. Per-user rate limits apply.	Charged per GB above 1,000 GB, so a payment method on file is likely needed. Check current billing.	Realtime pricing, TURN docs
Poly Haven assets	CC0, free for any use.	n/a	Poly Haven license

Unverified items (check current limits): - The Supabase Auth email rate limits on the built-in email sender. These are low; production needs a custom SMTP provider. Auth rate limits. - Whether `pg_cron` is on the free plan. - Exact free-plan CPU behavior for Durable Object invocations. The Durable Objects limits page lists 30 s of CPU per request, while the Workers Free plan lists 10 ms. Test bot compute against the free plan early.

13.2 Rough capacity on free tiers

These are estimates, not measurements. Assumptions: - A 2v2 game to 500 is about 8–12 hands. - Each hand is about 52 plays + 4 bids + a few chat/emote messages, so roughly **600–800 player-sent messages per game**. - Every action is saved as one overwritten state row. Turn alarms add about 1 row write each.

Resource	Usage per full game (est.)	Free daily limit	≈ Games/day
Durable Object rows written	~1,200–1,600 (state + alarms)	100,000	**~60–80** ← first bottleneck
Durable Object requests	~35 (messages ÷ 20) + 4 connects + alarm firings (~10–100)	100,000	~700–2,000
Worker requests	~4–10 (WebSocket upgrades, API calls)	100,000	thousands

Resource	Usage per full game (est.)	Free daily limit	≈ Games/day
Durable Object duration	small with hibernation	13,000 GB-s ≈ 28 active object-hours	hundreds+

Takeaways - **Rows written** are the first limit, by a wide margin. - Ways to stretch it: - Save state only when a trick ends or a bid is made (about 13 + 4 writes per hand instead of about 56). That costs a little replay logic: re-deal from the seeded hand and re-apply the current trick's plays. - Don't call `setAlarm` on every turn. Keep one alarm "ticking" (re-arm it only when it fires and check the real deadline then). That's roughly 1 write per 20 s of play instead of 1 per move. - Together these should push the ceiling to roughly **250–400 games/day**. Measure it in Phase 2 with real logs. - **Supabase Realtime's 200 concurrent connections** caps how many players can be online at once with chat/presence. At launch that's fine. Past that, move presence and DMs onto Durable Objects, or upgrade. - **Important timing gotcha:** Cloudflare's free daily limits reset at **00:00 UTC = 8:00 PM ET during daylight saving time (7:00 PM ET in winter)**. That's US prime time. If you hit a daily cap, the game breaks for everyone during the busiest hours until the reset. **Upgrade to Workers Paid before any marketing push.**

13.3 Where costs start

Trigger	First paid step	Published price
Daily Worker/Durable Object request or row-write caps hit, or you need more Durable Object CPU	**Workers Paid**	**$5/month minimum.** Includes 10M Worker requests/month (+$0.30/M), 30M CPU-ms (+$0.02/M), Durable Objects 1M requests (+$0.15/M), 400,000 GB-s (+$12.50/M GB-s), and SQLite rows (25B read and 50M written included, then $0.001/M read and $1.00/M written). Storage is 5 GB-month included, then $0.20/GB-month. (source)
More than 200 concurrent Realtime users, DB over 500 MB, egress over 5 GB, need backups, or need to stop the 7-day pausing	**Supabase Pro**	**From $25/month** per org, including $10 compute credit (covers one Micro project). Includes 100,000 MAU, 8 GB disk, 250 GB egress, Realtime 500 concurrent connections + 5M messages (then $10 per 1,000 connections and $2.50 per million messages). The spend cap is on by default. (source)
Video TURN relay above 1,000 GB/month	Cloudflare Realtime	$0.05/GB (source). Example estimate: a relayed player receiving 3 streams at ~0.5 Mbps is about 0.7 GB/hour, so 1,000 GB is roughly 1,400 relayed player-hours a month.
Video rooms bigger than 4 people, or recording	SFU / media server	Cloudflare Realtime SFU shares the same $0.05/GB and the same 1,000 GB free allowance.
iOS App Store	Apple Developer Program	**$99/year** (source)
Google Play	Play Console registration	**$25 one-time** (source)
Custom domain	Domain registrar	Typically ~$10–15/year (check current pricing). Optional at launch: `*.pages.dev` / `*.workers.dev` are free.
Transactional email (magic links) at volume	Custom SMTP provider	Many providers have free tiers; check current limits.

Rough monthly cost at moderate scale (estimate): Workers Paid $5 + Supabase Pro $25 ≈ **$30/month** covers a small but real community. Durable Object duration is the cost to watch, which hibernation keeps low.

---

14. Phased roadmap

Time estimates assume one developer working part-time to full-time. They're rough and meant for planning only.

Phase 1: Rules engine, tests, 2D bot prototype (≈ 3–5 weeks)

- `packages/rules`:
  - All three modes, with the full `TableRules` config and validation.
  - Seeded RNG so any game can be replayed.
  - Legal-move generation, trick resolution, bidding, and scoring including Nil, Blind Nil, bags, floor, and target.
- Tests:
  - Unit tests for every rule and house rule.
  - Property-based tests (fast-check): every card is dealt exactly once; scores add up; a legal move always exists; no illegal move is accepted.
  - Golden-file tests of complete scored games.
- `packages/bots`: heuristic bots at Easy and Normal.
- `apps/web`: a 2D table playable offline against 3 bots, with a score pad and house-rule picker.
- **Done when:** you can play full Joker, Cutthroat (3p and 4p), and Ace Down games against bots, and ≥95% of the rules code is covered by tests.

Phase 2: Online 2v2 with accounts (≈ 4–6 weeks)

- Supabase project with auth (magic link + Google) and a `profiles` table with RLS.
- Worker + Table Durable Object: WebSocket hibernation, server-authoritative play, turn timers through alarms, reconnection, bots taking over seats, and state saved to SQLite.
- Private tables (invite link or code) and a simple quick match (Lobby Durable Object). Match results written to Supabase.
- Logging and usage dashboards. **Measure real per-game Durable Object rows, requests, and duration** against the Section 13.2 estimates.
- **Done when:** four people on different networks can finish a 2v2 Joker game, and refreshing or switching networks mid-game rejoins cleanly.

Phase 3: Friends, chat, ratings, and leaderboards (≈ 4–6 weeks)

- Friend requests, presence, DMs, table chat, emotes.
- Block/report, profanity filter, age gate and minor restrictions.
- Glicko-2 per mode (`packages/rating`), stats, global and local leaderboards, region picker. Seasons optional.
- Cutthroat and Ace Down available online, in quick match and private tables.
- **Done when:** a new user can sign up, add a friend, chat, play rated games, and see themselves on the leaderboards.

Phase 4: Photoreal 3D (≈ 6–10 weeks, mostly art)

- Blender table and card models, a card face atlas, Poly Haven HDRI and textures, KTX2/Draco pipeline.
- R3F scene, quality tiers, auto-detect, card animations, and the 2D Classic fallback.
- Test on real low-end Android phones and older iPhones.
- **Done when:** steady 60 fps on mid-range phones at Medium, ≥30 fps on Low, first load at Low under ~10 MB.

Phase 5: Live video (≈ 3–5 weeks)

- WebRTC mesh, signaling through the Table Durable Object, STUN plus Cloudflare TURN with short-lived credentials.
- Adaptive quality, camera/mic controls, 18+ gating, video report flow.
- **Done when:** four players on mixed networks (including one on cellular or a strict corporate network through TURN) can see each other at a playable frame rate without hurting the game.

Phase 6: Native iOS/Android (≈ 3–6 weeks)

- Capacitor wrapper, Sign in with Apple, push notifications (FCM/APNs) for friend invites and "your turn" when backgrounded, deep links for invites, camera permissions, in-app account deletion.
- Store listings, privacy labels, and age ratings. Budget $99/year (Apple) + $25 once (Google).
- **Done when:** both apps are approved and in their stores, with playable performance in WebViews.

---

15. Open questions

1. **Ace Down rules:** besides "52 cards, no jokers, aces high," does Ace Down differ in bidding (minimum bids, no Nil, "board" bids), scoring (bags, target score), or anything special about aces? Is it played as partners or individually? *Default for now: 2v2 partners, standard Spades bidding and scoring (Section 3.1).*
2. **Default Joker house rules for ranked play:** which two cards get removed (default 2♥ and 2♦)? Should the 2♠ rank just under the jokers by default (default: no)? Do any of your tables also promote the 2♦?
3. **Cutthroat deck:** should Cutthroat use the Joker deck (default) or a standard 52? For 3 players, which cards come out?
4. **Default target score and floor:** 500 / −200 for partners and 300 for Cutthroat? Is Blind Nil allowed only when behind by 100?
5. **Nil bidder's tricks:** count as bags only (default), or toward the partner's bid?
6. **Card art:** commission original faces and backs, design them in-house, or license a deck? Is a four-color deck option wanted? This affects both branding and legal clearance.
7. **Name and branding:** game name, domain, and whether to buy a domain at launch (about $10–15/year) or launch on free `pages.dev` / `workers.dev` subdomains.
8. **Age policy and legal:** confirm 13+ accounts with minor restrictions and 18+ for video. Get Terms and a Privacy Policy written or reviewed by a lawyer before public launch.
9. **Rated games with friends:** should friends be allowed to play rated 2v2 together as partners against random opponents (common in Spades apps)? If so, Cutthroat stays strangers-only to limit collusion.
10. **Monetization (later):** cosmetics (card backs, tables), ads, or a supporter subscription? This decides when paid tiers make sense and what the app store review needs.
11. **Bots in rated games:** keep bot-filled games unrated (default), or rate the human players against a fixed bot rating?
12. **Seasons:** launch with seasons, or add them once there's an active player base?
13. **Spectators and replays:** wanted? This affects how much match history is stored, against the 500 MB Supabase cap.

Appendix A: Reference links

- Cloudflare Workers pricing: https://developers.cloudflare.com/workers/platform/pricing/

- Cloudflare Workers limits: https://developers.cloudflare.com/workers/platform/limits/
- Durable Objects pricing: https://developers.cloudflare.com/durable-objects/platform/pricing/
- Durable Objects limits: https://developers.cloudflare.com/durable-objects/platform/limits/
- Durable Objects WebSocket hibernation: https://developers.cloudflare.com/durable-objects/best-practices/websockets/
- Cloudflare Pages limits: https://developers.cloudflare.com/pages/platform/limits/
- Pages Functions pricing (static requests free): https://developers.cloudflare.com/pages/functions/pricing/
- Cloudflare Realtime pricing (SFU/TURN): https://developers.cloudflare.com/realtime/sfu/pricing/
- Cloudflare TURN service: https://developers.cloudflare.com/realtime/turn/
- Supabase pricing: https://supabase.com/pricing
- Supabase Realtime limits: https://supabase.com/docs/guides/realtime/limits
- Supabase Auth rate limits: https://supabase.com/docs/guides/auth/rate-limits
- Poly Haven license (CC0): https://polyhaven.com/license
- Glicko-2 (Glickman): http://www.glicko.net/glicko/glicko2.pdf
- Capacitor: https://capacitorjs.com/
- Apple Developer Program fee: https://developer.apple.com/programs/enroll/
- Google Play registration fee: https://support.google.com/googleplay/android-developer/answer/6112435
- Apple App Review Guidelines (UGC 1.2): https://developer.apple.com/app-store/review/guidelines/#user-generated-content
- FTC COPPA guidance: https://www.ftc.gov/business-guidance/privacy-security/childrens-privacy